CESDE MEDELLÍN · ESCUELA DE TECNOLOGÍAS Y COMUNICACIONES

💻 PROGANADO SAAS: ARQUITECTURA ESTRUCTURAL DEL FRONTEND

Modelado Modular de Componentes en React 19, Maquetación Declarativa con Tailwind CSS y Transición desde JavaScript Monolítico

ASIGNATURA: Introducción a la Programación (Momento 2 & 3)

DOCENTE TITULAR: Ing. Jonathan Felipe Piedrahita Nieto

EQUIPO PROGANADO: J. Gutiérrez, E. Villanueva, S. Correa, H. Pinto

AULA / MODALIDAD: Aula 406 · Tecnologías de Software

1. Resumen Ejecutivo y Diagnóstico Técnico

Para la sustentación de la plataforma **ProGanado SaaS** en la asignatura *Introducción a la Programación*, el equipo evaluó dos aproximaciones técnicas de frontend desarrolladas internamente:

- **Aproximación A (Desarrollada por Emilio Villanueva):** Aplicación completa local basada en HTML5, CSS3 monolítico tradicional (50 KB) y un archivo JavaScript único imperativo (`proganado.js` con más de 1.950 líneas) apoyado en un microservidor Python con SQLite.
- **Aproximación B (Arquitectura Moderna de Componentes):** Aplicación de Página Única (SPA) basada en **React 19**, maquetación modular con **Tailwind CSS utility-first** y empaquetador ultrarrápido **Vite**.

Decisión Estratégica Adoptada por el Equipo: Adoptamos el **100% del diseño visual, los flujos operativos, las fichas zootécnicas y los módulos ganaderos concebidos por Emilio**, pero **refactorizamos y transformamos su código a componentes funcionales de React y clases de Tailwind CSS**. Esto nos permite conservar la riqueza zootécnica de Emilio cumpliendo con los estándares de ingeniería y la rúbrica pedagógica del CESDE.

2. Cuadro Comparativo: ¿Cuál Frontend es Mejor y Por Qué?

CRITERIO TÉCNICO	FRONTEND MONOLÍTICO (EMILIO / VANILLA JS)	FRONTEND REFACTORIZADO (REACT 19 + TAILWIND)
Diseño Visual y UX	Excelente y muy completo. Paleta ganadera sobria, fotos reales de vacas y 7 módulos completos.	**Idéntico visualmente:** Se reutilizaron los mismos logos, banners y fotos en `public/assets/`.
Modularidad (SRP)	Monolito Todo en 1 archivo (`proganado.js`, 103 KB, ~2.000 líneas). Difícil de mantener en equipo.	Modular Dividido en 10 componentes independientes (<250 líneas cada uno) en `src/components/`.
Manipulación del DOM	**Imperativa:** `document.getElementById()`, concatenaciones de `innerHTML` y clases manuales. Propenso a errores.	**Declarativa:** El estado (`useState`) gobierna la interfaz. React reconcilia el Virtual DOM automáticamente.
Hojas de Estilo	1 archivo CSS de 50 KB con cientos de selectores estáticos. Riesgo de colisión de clases y especificidad.	Tailwind CSS Clases utilitarias directas en el JSX. Cero CSS huérfano; bundle final optimizado de 26 KB.
Evaluación CESDE	Demuestra lógica de negocio, pero no evidencia el uso de frameworks modernos ni separación de componentes.	**Máxima calificación:** Evidencia dominio de JSX, hooks, props, Tailwind y empaquetado moderno con Vite.

3. Árbol Jerárquico de Componentes (Component Tree)

La estructura de ProGanado en React sigue el principio de diseño de **Unidirectional Data Flow (Flujo Unidireccional de Datos)**. La información desciende de padres a hijos mediante `props`, y los eventos ascienden mediante funciones de callback:

```
index.html (Contenedor raíz <div id="root"></div>) └── main.jsx (Punto de entrada de React 19: createRoot) └── App.jsx (Orquestador Principal y Almacén de Estado Centralizado) ├── Navbar.jsx (Barra de Navegación Institucional con 8 Tabs + Alertas ICA) ├── Main Content Router (Renderizado Condicional por activeTab) │ ├── [dashboard] ➔ DashboardView.jsx (KPIs, Donut Chart, Agenda y Acopio) │ ├── [inventory] ➔ InventoryView.jsx (Grid de Ganado, Buscador y Filtro) │ ├── [health] ➔ HealthModule.jsx (Alerta de Retiro ICA y Medicamentos) │ ├── [reproduction]➔ ReproductionModule.jsx (Gestación, Celos y Partos) │ ├── [production] ➔ ProductionModule.jsx (Pesajes de Leche y Entregas Colanta) │ ├── [pastures] ➔ PasturesModule.jsx (Rotación de Potreros PRV de Voisin) │ ├── [accounting] ➔ AccountingModule.jsx (Facturas, Entradas y Salidas) │ └── [users] ➔ UsersModule.jsx (Control de Roles y Verificación 2FA) ├── Footer.jsx (Metadatos institucionales y créditos del equipo) └── Modals (Superposiciones Reactivas Fuera del Flujo) ├── RegisterModal.jsx (Formulario controlado para nuevo bovino) └── CowDetailModal.jsx (Ficha médica y hoja de vida individual)
```

4. Los 4 Pilares de Programación Aplicados en la Refactorización

Pilar 1: JSX (JavaScript XML) — El Lenguaje de la Interfaz

En lugar de armar cadenas de texto peligrosas con `innerHTML = '<div class="card">' + data + '</div>'` como hacía el JavaScript tradicional, en React escribimos **JSX**: una sintaxis que permite mezclar la lógica de JavaScript con la estructura HTML de forma segura y tipada.

```
// Ejemplo en ProGanado (Renderizado Seguro en JSX):
return (
  <div className="p-4 rounded-2xl bg-white border border-[#e2ece6] shadow-sm">
    <h4 className="font-bold text-sm text-[#2b3b32]">{vaca.nombre}</h4>
    <span className="text-xs text-[#198754]">{vaca.litrosDia} L / día</span>
  </div>
);
```

Pilar 2: Estado Reactivo con el Hook `useState`

En la versión de Emilio, si se agregaba una vaca, había que llamar manualmente a una función que borraba la tabla del DOM y la volvía a dibujar con `tbody.innerHTML = ''`. En React, la vista es una función pura del estado. Cuando el estado cambia con `setBovinos()`, React recalcula y actualiza **únicamente la fila modificada** sin parpadeos.

```
// Estado centralizado en App.jsx con persistencia atómica en localStorage:
const [bovinos, setBovinos] = useState(() => {
  const saved = localStorage.getItem('proganado_herd_v3');
  return saved ? JSON.parse(saved) : INITIAL_HERD;
});

useEffect(() => {
  localStorage.setItem('proganado_herd_v3', JSON.stringify(bovinos));
}, [bovinos]);
```

5. Sistema de Estilizado Moderno: ¿Por Qué Tailwind CSS?

El Problema del CSS Tradicional (La Trampa de los 50 KB): En el archivo `proganado.css` de Emilio, existen más de 1.200 líneas de reglas globales como `.card`, `.btn-primary`, `.metric-icon`. Si dos compañeros creaban una clase con el mismo nombre, los estilos chocaban y rompían la interfaz. Además, para saber qué hacía un botón, había que saltar de un archivo a otro.

La Solución con Tailwind CSS (Utility-First): Con Tailwind, estilizamos directamente dentro del componente usando clases utilitarias predecibles. Esto garantiza que cada botón, tarjeta o tabla tenga su diseño autocontenido y no afecte al resto de la página.

Tokens de Diseño y Paleta Ganadera Oficial

CLASE / TOKEN TAILWIND	VALOR HEX / CSS	USO ZOOTÉCNICO EN PROGANADO
`bg-[#0b291b]`	`#0b291b` (Verde Bosque Profundo)	Navbar superior, encabezados de tablas y banners de identidad institucional.
`bg-[#198754]` / `text-[#198754]`	`#198754` (Verde Esmeralda Agro)	Botones primarios, indicadores de inocuidad 100%, promedios lecheros óptimos.
`bg-[#fff3cd]` / `text-[#664d03]`	`#fff3cd` (Ámbar Alerta)	Alerta sanitaria ICA: Animales en tratamiento médico con leche suspendida.
`bg-[#f8d7da]` / `text-[#842029]`	`#f8d7da` (Rojo Alerta)	Salidas de dinero, celos próximos a vencer y periodos críticos de retiro.
`rounded-2xl border-[#e2ece6]`	Border radius 1rem + borde suave	Tarjetas modulares estilo Dashboard moderno (Bento Grid).

6. Desglose Módulo por Módulo (Los 8 Módulos de Emilio en React)

Módulo 1: Inicio (Panel de Control y Resumen del Hato)

Archivo React: `src/components/DashboardView.jsx`

Centraliza la operación completa de la finca La Pradera. Muestra 4 tarjetas de métricas calculadas en tiempo real mediante programación funcional (`Array.prototype.reduce` y `filter`):

- **Total Ganado:** Conteo dinámico de la población bovina.
- **En Gestación:** Número de vacas con diagnóstico de preñez confirmado.
- **En Retiro ICA:** Alerta crítica de vacas tratadas con fármacos (cero leche a tanque).
- **Litros Hoy:** Sumatoria automática de la producción en turnos AM y PM.
- **Gráfico Donut:** Composición porcentual del hato por raza y estado fisiológico.

Módulo 2: Ganado (Inventario y Ficha Individual)

Archivo React: `src/components/InventoryView.jsx` y `CowDetailModal.jsx`

Permite visualizar el hato completo en tarjetas interactivas con foto de cada animal (respetando las fotos tomadas en finca de la carpeta `assets/ganado/`). Incluye barra de búsqueda instantánea por arete o nombre y filtro reactivo por estado (En Ordeño, Seca, Novilla, Toro). Al hacer clic en un animal, se dispara el modal de **Hoja de Vida** con genealogía, potrero, días de lactancia y peso.

Módulo 3: Salud (Días de Retiro ICA y Farmacopea)

Archivo React: `src/components/HealthModule.jsx`

Implementa la regla de negocio más importante de la industria lechera: **La Inocuidad y Prevención de Multas**. Si una vaca recibe un antibiótico como *Oxitetraciclina LA* (6 días de retiro) o *Cefalexina* (3 días), el sistema calcula automáticamente los días restantes y tiñe la ficha del animal en amarillo de alerta ("Cinta Roja"), impidiendo que esa leche contamine el tanque de frío de Colanta.

Módulo 4: Reproducción (Gestación, Celos y Partos)

Archivo React: `src/components/ReproductionModule.jsx`

Controla el ciclo reproductivo de las vacas para evitar el problema de los "días abiertos" prolongados (vacas que pasan meses sin preñarse y generan pérdidas económicas). Registra fechas de servicio, inseminaciones con pajillas importadas, cálculo de la Fecha Probable de Parto (FPP = Fecha de servicio + 283 días) y nacimientos.

Módulo 5: Producción (Ordeño Mecánico y Liquidación Colanta)

Archivo React: `src/components/ProductionModule.jsx`

Registra los pesajes diarios de leche por vaca y concilia las entregas al carro cisterna de Colanta. Almacena el volumen despachado en litros, la liquidación estimada en pesos colombianos y el resultado microbiológico de laboratorio (recuento de Unidades Formadoras de Colonias - UFC/ml), certificando la bonificación por calidad higiénica Grado A.

Módulo 6: Potreros PRV (Pastoreo Racional Voisin)

Archivo React: `src/components/PasturesModule.jsx`

Aplica las leyes universales del científico André Voisin para la rotación regenerativa de pasturas. Muestra el estado en tiempo real de los potreros (Norte, Sur, Este y La Colina), alertando si un lote supera el tiempo máximo de ocupación (1 a 2 días para evitar que el ganado coma el rebrote) y midiendo los días de descanso acumulados (30 a 42 días) para que la hierba alcance su Punto Óptimo de Reposo.

Módulo 7: Contabilidad (Libro Financiero y Facturas con Foto)

Archivo React: `src/components/AccountingModule.jsx`

Gestiona el flujo de caja operativo de la finca. Registra ingresos (liquidaciones de Colanta, venta de terneros) y egresos (bultos de concentrado lechero, sales mineralizadas, medicamentos veterinarios). Cada registro cuenta con su comprobante digital adjunto para una auditoría contable transparente.

Módulo 8: Usuarios & Seguridad 2FA

Archivo React: `src/components/UsersModule.jsx`

Implementa la arquitectura de control de acceso basada en roles (RBAC): Administradores con control total y Usuarios Operativos de consulta y campo. Incluye el simulador del protocolo de **Autenticación en Dos Pasos (2FA)** con tokens de 6 dígitos para blindar el acceso a la base de datos de la finca.

7. Cumplimiento de la Rúbrica de Evaluación CESDE

CRITERIO EVALUADO	EVIDENCIA CONCRETA EN PROGANADO (REACT + TAILWIND)	ESTADO DE VERIFICACIÓN
Modularidad y Limpieza de Código	10 componentes separados en `src/components/` bajo el principio de responsabilidad única (SRP < 300 líneas por archivo).	✅ 100% Verificado (0 errores de compilación)
Lógica de Programación	Uso de funciones flecha, desestructuración, operadores ternarios, mapeo de listas con key y filtrado funcional.	✅ 100% Verificado
Manejo del Estado	Hook `useState` para persistencia local y sincronización reactiva bidireccional en tiempo real.	✅ 100% Verificado
Diseño y Maquetación	Diseño responsivo móvil y escritorio con Tailwind CSS, Flexbox, CSS Grid y paleta cromática verde esmeralda.	✅ 100% Verificado
Construcción y Despliegue	Empaquetado optimizado para producción con Vite (`npm run build`) generando bundle ultraligero de 26 KB.	✅ 100% Verificado (Build exitoso)

8. Aprobación y Firmas del Equipo de Proyecto

ENTREGA FORMAL DE COMPONENTE FRONTEND — PROGANADO SAAS

Escuela de Tecnologías y Comunicaciones CESDE · Medellín 2026

CALIFICACIÓN: [/ 5.0]

_____	_____	_____	_____
Jeiser Abraham Gutiérrez Torres	**Emilio Villanueva**	**Sebastián Correa**	**Humberto Pinto**
Líder Técnico & Arquitectura	Diseño UI/UX & Flujos	Modelado de Datos & Backend	Zootecnia & Sanidad Animal

Docente Evaluador: **Ing. Jonathan Felipe Piedrahita Nieto**

Firma Evaluador: _____